

Projeto PW-3

React + Vite + TypeScript



ETAPA 1 — Fundamentos + Estrutura inicial

Listar veículos (GET)



Objetivo da aula

- Criar estrutura básica;
- Listar veículos (GET);
- Separar minimamente responsabilidades (sem “exagero” ainda);



NÃO vamos começar usando ainda:

- Axios ✗
- React Query ✗
- Zod ✗
- React Hook Form ✗
- Context API ✗

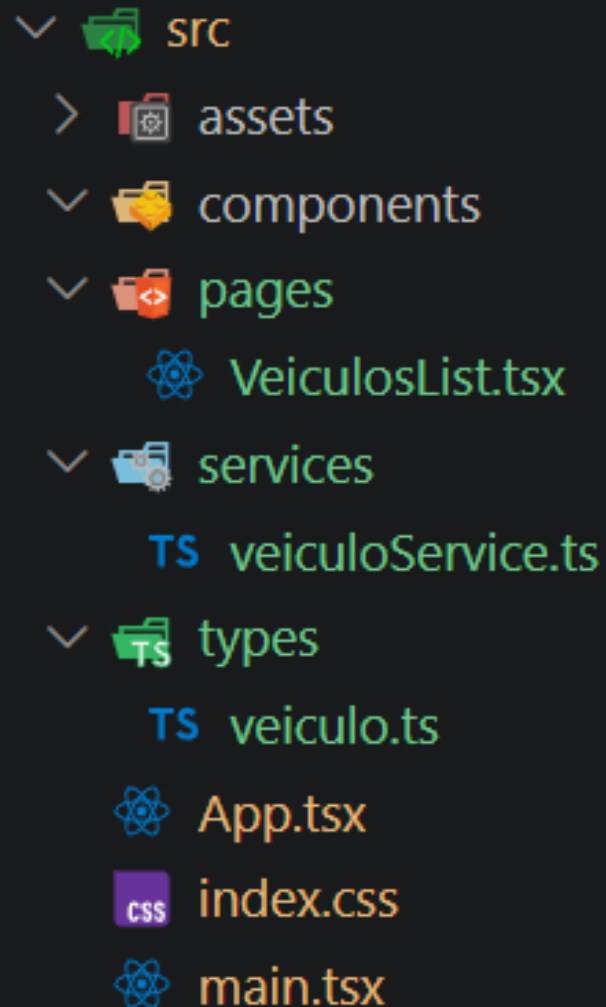


Vamos começar simples (mas já do jeito certo).

1. Criar projeto

- Comando via cmd
 - `npm create vite@latest`
- Defina as configurações solicitadas para:
 - ✓ Project-name: APP
 - ✓ Package name: app
 - ✓ Select a framework: React
 - ✓ Select a variant: TypeScript
 - ✓ Install with npm and start now?: yes
- Depois:
 - `cd nome-do-projeto`
`npm install`
`npm run dev`

2. Estrutura inicial (simples, propositalmente)



```

src
├── assets
├── components
├── pages
│   └── VeiculosList.tsx
├── services
│   └── TS veiculoService.ts
├── types
│   ├── TS veiculo.ts
│   ├── App.tsx
│   ├── index.css
│   └── main.tsx

```

The image shows a file explorer interface with a dark background. The 'src' directory is expanded, showing a tree structure. The 'assets' directory is collapsed. The 'components' directory is collapsed. The 'pages' directory is expanded, showing 'VeiculosList.tsx'. The 'services' directory is expanded, showing 'TS veiculoService.ts'. The 'types' directory is expanded, showing 'TS veiculo.ts', 'App.tsx', 'index.css', and 'main.tsx'. Each file and directory has a small icon next to it, such as a folder icon for directories and a TypeScript icon for .ts files.

3. Tipagem (primeiro contato com arquitetura)

- src/types/veiculo.ts

```
1 export interface Veiculo {  
2     id: number;  
3     Anunciantes_id: number;  
4     Fabricantes_id: number;  
5     modelo: string;  
6     descricao: string;  
7     valor: number;  
8     ano: number;  
9     ano_modelo: number;  
10    fotos: string[];  
11 }
```

4. Service (primeira separação importante)

- src/services/veiculoService.ts

```
1 import type { Veiculo } from "../types/veiculo";
2
3 const API_URL = "http://localhost:3000/veiculos";
4
5 export async function getVeiculos(): Promise<Veiculo[]> {
6     const response = await fetch(API_URL);
7     return response.json();
8 }
```

5. Página (primeiro consumo de dados)

```
1 import { useEffect, useState } from "react";
2 import { getVeiculos } from "../services/veiculoService";
3 import type { Veiculo } from "../types/veiculo";
4
5 export function VeiculosList() {
6   const [veiculos, setVeiculos] = useState<Veiculo[]>([]);
7
8   useEffect(() => {
9     getVeiculos().then(setVeiculos);
10  }, []);
11
12  return (
13    <div>
14      <h1>Lista de Veículos</h1>
15
16      {veiculos.map((veiculo) => (
17        <div key={veiculo.id}>
18          <h2>{veiculo.modelo}</h2>
19          <p>{veiculo.descricao}</p>
20          <strong>R$ {veiculo.valor}</strong>
21        </div>
22      ))}
23    </div>
24  );
25 }
```

- src/pages/VeiculosList.tsx

6. App

- src/App.tsx

```
1 import { VeiculosList } from "../pages/VeiculosList";
2
3 export default function App() {
4   return <VeiculosList />;
5 }
```



O que vimos nessa aula



Conceitos-chave:

- ✓ useEffect
- ✓ useState
- ✓ Tipagem com TypeScript
- ✓ Separação básica (service vs UI)
- ✓ Consumo de API
- ✓ Estrutura inicial

Problemas INTENCIONAIS

- × Fetch dentro do service ainda é simples demais;
- × Sem tratamento de erro;
- × Sem loading;
- × Sem reuso de UI;
- × Tudo ainda meio acoplado;
- × Sem uso de ferramentas modernas;

Exercícios para os alunos

- Lista

1. Exibir ano e ano_modelo;
2. Mostrar quantidade de fotos;
3. Criar componente VeiculoCard;

- Desafio

1. Mostrar apenas veículos acima de R\$ 80.000;

O que vem na próxima etapa (spoiler)

- 👉 Componentização de verdade;
- 👉 Separar VeiculoCard;
- 👉 Introduzir props corretamente;
- 👉 Melhorar organização;
- 👉 Começar a reduzir acoplamento